Robot Heatmap Localization

Lirui Wang
University of Washington
Seattle, USA
liruiw@uw.edu

***Abstract*—In this project, we attempt to predict a heatmap of robot's location and heading given robot's current laser scan measurement. We discretize the states and treat the problem as classification with a standard encoder-decoder network, similar to many image segmentation task. Taking the full map and an explicit depth map representation as input, our model is able to successfully localize the robot when the laser scan provides enough information. Our code can be found here and video can be found here.**

I. Introduction

Localization remains one of the most challenging problem in robot navigation. In 2D occupancy grid map setting, the task is to predict robot state $[x, y, \theta]$ given an occupancy map M of the environment and laser scan measurement $d_1, d_2, ..., d_N$ where N denotes the number of laser rays available. We treat this task as a classification problem, where each class is one of the candidate robot states. We discretize θ to $\theta_1, ..., \theta_K$ where $K = 4$ denotes the region of $[0, \pi/2, \pi, \pi/3]$. To simulate the real world scenario, our robot only has a field of view of $\frac{2}{3}\pi$. The model output is the belief over the state space. The key challenge for learning localization in this case is to find a good representation to jointly encode the map and depth. Many related works such as [1] preprocess the map by extract the features, and cast localization as a feature matching problem. There are potentially two problems. To admit a reasonable fine discretization, the map or the state space is assumed to be small, otherwise the local feature storage can take up lots of space. The second problem is that one has to do a preprocessing step for the test map, if unknown in advance, which can take lots of time. While it seems not difficult to do one-shot localization with heatmap for neural network, the high dimensions of input and output make the problem nontrivial.

II. Methodology

A. Representation

Both the map and the laser scan are necessary information to localize the robot. The occupancy map x_{ijk} is naturally represented as a binary image, rescale to 128×128, where 0 denote the occupied and 1 denote free space. i, j denote the image coordinate and k denote the channel coordinate. Our network f_θ is a standard convolutional neural network parametrized by θ to extract features from images. While a naive depth encoding is to simply concatenate the M dimensional vector to with image features, this can pose the learning problem too hard. The network would then have to learn what laser scan scalar means in terms of obstacle to be used with the input map. This in fact shows barely any meaningful signal in our experiments. Instead, we represent laser scans as a obstacle map at a "canonical state": the map origin with 0 degree orientation. While the information can differ from map to map in practice, it can be used to effectively represent the depth. Specifically, we compute the position of the observed obstacle with the input laser scan value, and create an obstacle map of the same size as input image. Shown on Fig 1, this "canonical state" obstacle map has 1 everywhere except where the obstacles locate at if the robot is at the "canonical state". In this way, the network is only required to learn the rotation and translation to match the obstacle map with every possible state in the input image. While a finer way can be to convolute the centered obstacle map with the input map, we simply concatenate these two maps of the same size as network input.

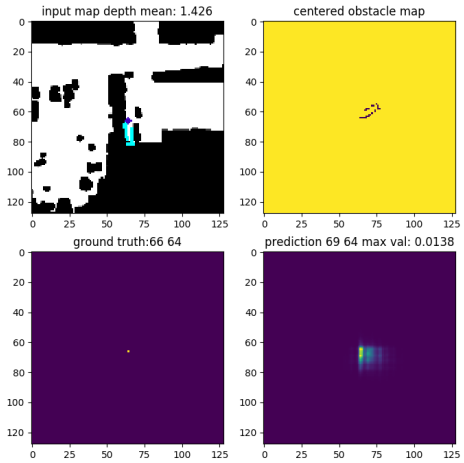

Fig. 1: Visualization of Input and Output. Top Left) Input depth map, where the colored marker denote the robot location with certain orientation, and the cyan dots denote the observed obstacles from laser scan. Top Right) The "canonical state" observation. Bottom Left) The groundtruth robot state ignoring orientation. Bottom Right) The prediction heatmap which centers around the groundtruth state.

The output y_{ijk} is a heatmap of the same size with channel number K representing the probability of robot state at each

state location and orientation. To map network output to probability, we compute a softmax over all states

$$z_{ijk} = \frac{\exp(y_{ijk})}{\sum_{i,j,k} \exp(y_{ijk})}$$

The training loss is a standard binary cross entropy loss where the groundtruth y^* is 1 at the actual robot state. Moreover, to have a proper scale over the heatmap softmax, we have an implicit regularization that penalizes large scale output. Our overall loss then looks like

$$L = \sum_{i,j,k} -(y^*_{ijk} \log(z_{ijk}) + (1 - y^*_{ijk}) \log(1 - z_{ijk})) + \gamma\sigma(y_{ijk}) \tag{1}$$

where γ is a weight scalar and and σ is the sigmoid function.

B. Training

We split the 100 maps into 90 training maps and 10 test maps. The groundtruth robot state is randomly sampled from free space and the orientation is randomly sampled from $[0, 2\pi]$. To minimally represent the map and enable better generalization, we crop the full map around the robot state with margin as the laser scan range. Besides the naive feature concatenation of depth vector, we also experiment with separately processing obstacle map and input map and then concatenate. We also experiment with spatial softmax [2] for an explicit learning of attention at robot state. We use Adam as our optimizer and an initial learning rate of 0.001 with step decay. During training, we often observe a slow but small convergence. Since our robot does not have full 360 field of view, our learned network can still suffer from ambiguous observation, as shown in 2. Worse scenario happens when the robot faces the near wall, then the obstacle observation can be completely ambiguous and we suspect the network output only comes from memorization of the map.

C. Testing

At testing time, we need to predict the robot state over the full map. To mimic the same cropping size as training, we need to extract patches from the map, run our model on each patch and stitch the results together. In practice, we would stitch $\bar{y} = \cup_p y^p_{ijk}$ where p denotes the index of the map patch. The overall belief $\bar{z}$ is then a softmax over the full map $\bar{y}$. As shown in Fig.3, the robot would predict multimodal state distribution for those that look like a potential state. We empirically observe that the network works better in smaller image dimension, 64 rather than 128 and full 360 laser angles rather than partial 120 degrees, which confirm our natural understanding of the task difficulty. Moreover, it's nontrivial for the network training to remove all false positive in the full map prediction. While the groundtruth state can often be located in the corresponding cropped submap, sometimes other found false negatives can have a higher value and degrade the performance of full map prediction after softmax.

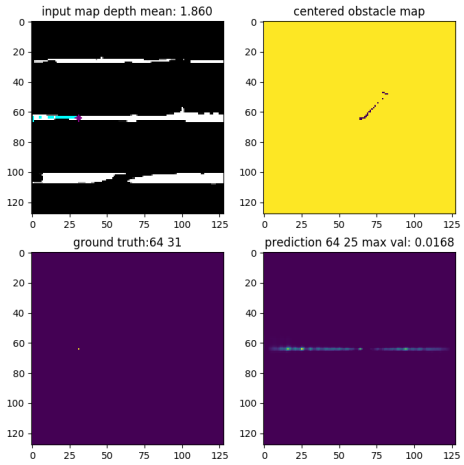

Fig. 2: The ambiguity inherently lies in localizing with partial field of view and a large map. With a upward facing orientation, the robot cannot localize itself any finer along the stripe in the middle.

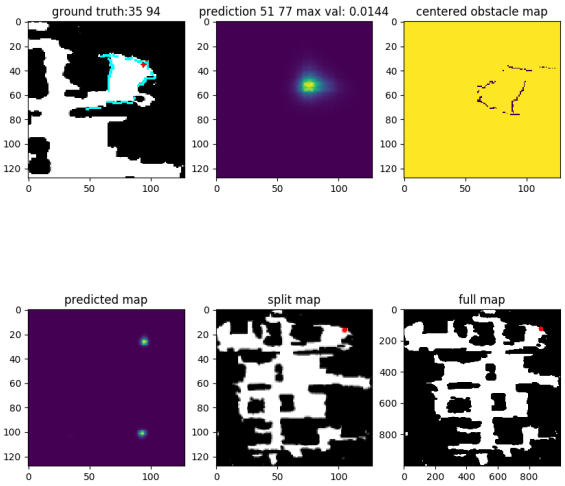

Fig. 3: The stitched full map prediction at test time. Since we have to crop the image and resize, the stitched map can look different from the actual full map.

III. Conclusion

Overall, we conduct a project over robot localization by heatmap inference over the grid map. Our encoder-decoder network effectively learns to predict robot state with an obstacle map representation of the laser scan. Since the performance for a higher resolution heatmap and partial laser scan is yet to be perfect, we are actively working to see if a potential solution can be found by adding more structure to the network.

References

[1] Devendra Singh Chaplot, Emilio Parisotto, and Ruslan Salakhutdinov. Active neural localization. *arXiv preprint arXiv:1801.08214*, 2018.

[2] Chelsea Finn, Xin Yu Tan, Yan Duan, Trevor Darrell, Sergey Levine, and Pieter Abbeel. Deep spatial autoencoders for visuomotor learning. In *2016 IEEE International Conference on Robotics and Automation (ICRA)*, pages 512–519. IEEE, 2016.